

객체 스토리지로 파일 저장하기

학습 체크리스트 (1/2)

☐ 로컬 저장 방식의 한계와 객체 스토리지의 필요성 이해

☐ 버킷·객체·키 개념과 객체 키 설계 원칙 습득

☐ Supabase 비공개 버킷과 서버 전용 S3 키 관리 방법 습득

☐ `endpointOverride` · `forcePathStyle` 로 S3 연결을 구성하는 방법 이해

학습 체크리스트 (2/2)

☐ `FileStore` 구현만 교체해 저장소를 바꾸는 구조 이해

☐ `PutObjectRequest` · `deleteObject` 로 객체를 저장·삭제하는 흐름 습득

☐ 자체 컨트롤러로 비공개 객체를 읽어 응답하는 방법 습득

☐ 프리티어 한도와 업로드 실패·고아 객체 주의점 파악

로컬 저장의 한계와 객체 스토리지

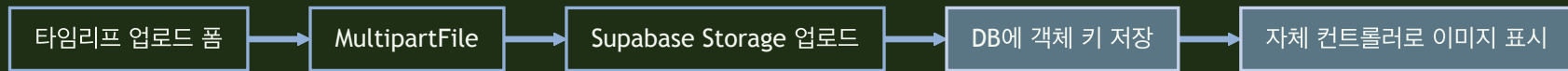
로컬 <code>static</code> 저장의 한계	객체 스토리지의 해결 방식
실행 JAR 내부에 새 파일을 쓸 수 없음	애플리케이션 밖의 저장 서비스에 보관
재배포할 때 기존 파일이 사라질 수 있음	배포와 파일의 수명 주기를 분리
서버마다 서로 다른 파일이 남음	모든 서버가 같은 저장소를 공유

객체 스토리지 3가지 용어

용어	의미	예
버킷 (bucket)	파일을 묶는 최상위 저장 공간	<code>course-images</code>
객체 (object)	버킷 안에 실제로 저장된 파일	업로드된 이미지 파일
키 (key)	버킷 안에서 객체를 찾는 이름	<code>profiles/uuid.png</code>

- **이름 전체가 키**: 슬래시를 포함해도 실제 디렉터리가 아니라 이름 전체가 하나의 키
- **규칙으로 그룹화**: 키를 정하는 규칙만 정해두면 폴더처럼 묶어서 다룰 수 있음

업로드부터 화면 표시까지의 흐름



Supabase Free Plan 주요 한도

항목	한도
파일 저장 공간	1GB
파일 하나 최대 크기	50MB
캐시된 전송량	5GB
그 외 전송량	5GB
이미지 변환	미제공

프리티어에 맞춘 실습 정책

- **업로드 용량 제한:** 이미지 하나당 5MB로 제한해 공간과 전송량을 절약
- **이미지 변환 제외:** Free Plan에서 제공하지 않는 리사이징·최적화는 다루지 않음
- **프로젝트 일시 중지:** Free 프로젝트는 1주 미사용 시 일시 중지될 수 있음

업로드 용량 제한 설정

```
1  spring:
2    servlet:
3      multipart:
4        max-file-size: 5MB
5        max-request-size: 20MB
```

버킷과 S3 연결 정보 준비 순서

단계	할 일
①	Storage에서 <code>course-images</code> 버킷 생성
②	버킷을 비공개(private)로 설정
③	S3 연결 활성화 후 Access Key ID·Secret Access Key 발급
④	같은 화면의 endpoint와 region 확인

비공개 버킷과 서버 전용 키

- 공개 URL 차단: 공개 URL만으로는 객체를 열 수 없음
- 서버에서 권한 판단: 자체 컨트롤러가 접근을 확인한 뒤 객체를 응답
- 강력한 S3 키: 모든 버킷에 접근하고 RLS를 우회하므로 서버에서만 사용
- 환경 변수 주입: HTML·JavaScript·Git에 넣지 않고 실행 환경에서 주입

S3 연결 정보와 마스킹 예시

항목	마스킹한 예시
Project ref	abcdefghijklmnpqrst
Bucket	course-images
Region	ap-northeast-2
Endpoint	https://abcdefghijklmnpqrst.storage.supabase.co/storage/v1/s3
Access key	*****
Secret key	*****

비밀 값은 실행 환경에서 주입

- 회수 불가: 소스 코드와 Git 이력에 한 번 남으면 되돌려 지우기 어려움
- 설정 파일 역할: `application.yml` 에는 환경 변수 이름만 적어둠
- 실제 값 주입: 값 자체는 실행 환경에서 주입해 소스와 분리함

설정 파일에는 환경 변수 이름만

```
1  app:
2    storage:
3      bucket: ${SUPABASE_S3_BUCKET}
4      endpoint: ${SUPABASE_S3_ENDPOINT}
5      region: ${SUPABASE_S3_REGION}
6      access-key: ${SUPABASE_S3_ACCESS_KEY}
7      secret-key: ${SUPABASE_S3_SECRET_KEY}
```

실행 환경에 넣는 값

```
1 SUPABASE_S3_BUCKET=course-images
2 SUPABASE_S3_ENDPOINT=https://abcdefghijklmnoqrst.storage.supabase.co/storage/v1/s3
3 SUPABASE_S3_REGION=ap-northeast-2
4 SUPABASE_S3_ACCESS_KEY=*****
5 SUPABASE_S3_SECRET_KEY=*****
```

Supabase에 AWS SDK를 쓰는 이유

- S3 호환 API: Supabase Storage가 S3와 동일한 방식의 API를 제공함
- 동일한 클라이언트 사용: Java에서는 AWS SDK for Java 2.x의 `S3Client`로 그대로 연결
- 최소 의존성: 필요한 모듈은 S3 하나뿐이며 다른 AWS 서비스 모듈은 불필요

AWS SDK 2.x의 S3 모듈 추가

```
1 dependencies {  
2     implementation("software.amazon.awssdk:s3:2.51.1")  
3 }
```

연결 정보를 묶는 StorageProperties

```
1 @ConfigurationProperties("app.storage")
2 public record StorageProperties(
3     String bucket,
4     String endpoint,
5     String region,
6     String accessKey,
7     String secretKey
8 ) {
9 }
```

Supabase 연결의 핵심 세 가지

- 엔드포인트 재정의: `endpointOverride` 로 AWS 대신 Supabase 주소를 사용하도록 덮어씀
- 자격 증명 지정: 발급받은 액세스 키·시크릿 키로 인증 정보를 구성
- path-style 강제: Supabase S3는 path-style 주소 방식이므로 `forcePathStyle(true)` 필요

S3Client 빈 생성

```
1  @Bean
2  S3Client s3Client(StorageProperties p) {
3      return S3Client.builder()
4          .region(Region.of(p.region()))
5          .endpointOverride(URI.create(p.endpoint()))
6          .credentialsProvider(StaticCredentialsProvider.create(
7              AwsBasicCredentials.create(p.accessKey(), p.secretKey())))
8          .forcePathStyle(true)
9          .build();
10 }
```

Spring Cloud AWS 스타터라는 선택지

- 자동 구성: `S3Client` 를 직접 만들지 않아도 스타터가 구성함
- 둘 중 하나만 선택: 직접 만든 `S3Client` 빈과 스타터 자동 구성을 중복 적용하지 않음
- 설정 중심 구성: endpoint와 path-style 접근도 `application.yml` 에서 지정

스타터 의존성

```
1 dependencies {  
2     implementation("io.awspring.cloud:spring-cloud-aws-starter-s3:4.1.0")  
3 }
```

스타터 방식의 설정

```
1  spring:
2    cloud:
3      aws:
4        credentials:
5          access-key: ${SUPABASE_S3_ACCESS_KEY}
6          secret-key: ${SUPABASE_S3_SECRET_KEY}
7      s3:
8        endpoint: ${SUPABASE_S3_ENDPOINT}
9        region: ${SUPABASE_S3_REGION}
10       path-style-access-enabled: true
```

직접 설정과 스타터 방식 비교

방식	특징
AWS SDK 직접 설정	연결 과정이 명확하고 Spring 밖에서도 재사용 가능
Spring Cloud AWS 스타터	<code>S3Client</code> · <code>S3Template</code> 자동 등록으로 설정이 간결함

- 두 방식을 중복 적용하지 않고 프로젝트 상황에 맞춰 하나만 선택
- 어느 방식을 택해도 `S3FileStore` 와 컨트롤러의 역할은 유지

S3Template으로 간단히 업로드

```
1 s3Template.upload(properties.bucket(), key, file.getInputStream());
```

저장 구현만 갈아끼우기

- 그대로 유지: 컨트롤러, `User`, `Book`, `BookImage`의 구조는 손대지 않음
- 교체 대상: 실제 저장을 수행하는 `FileStore`의 내부 구현만 바뀜
- 가능한 이유: 저장 방식을 `FileStore` 인터페이스 뒤로 숨겨 두었기 때문

저장 방식을 감추는 FileStore 인터페이스

```
1 public interface FileStore {  
2  
3     UploadFile store(MultipartFile file) throws IOException;  
4  
5     void delete(String storedName);  
6 }
```

DB에는 URL이 아닌 객체 키를 저장

항목	저장 값과 원칙
<code>originalName</code>	경로 부분을 제거한 사용자 표시용 이름
<code>storedName</code>	로컬에서는 파일명, 객체 스토리지에서는 객체 키
저장 예	<code>봄 표지.png</code> / <code>books/2f60c5a8-....png</code>
URL	endpoint·공개 URL은 저장하지 않고 조회할 때 생성

객체 키와 업로드 요청 만들기

```
1 String original = Objects.requireNonNull(file.getOriginalFilename());
2 String safeOriginalName = StringUtils.getFilename(StringUtils.cleanPath(original));
3 String key = "books/" + UUID.randomUUID() + "." + extension;
4 PutObjectRequest request = PutObjectRequest.builder()
5     .bucket(properties.bucket())
6     .key(key)
7     .contentType(validatedType.toString())
8     .build();
```

스트림으로 객체 업로드

```
1 try (InputStream input = file.getInputStream()) {  
2     s3Client.putObject(request,  
3         RequestBody.fromInputStream(input, file.getSize()));  
4 }  
5 return new UploadFile(safeOriginalName, key);
```

업로드 검증과 스트림 처리

- 검증 우선: 크기·확장자·파일 시그니처를 확인하고 안전한 미디어 타입 결정
- 스트림 사용: `getBytes()` 대신 스트림과 크기를 넘겨 바이트 배열 복제를 피함
- 정확한 길이: `fromInputStream`에는 실제 바이트 크기와 같은 값을 전달
- 키 명명: 사용자 파일명 대신 UUID 키를 써서 덮어쓰기를 방지

저장 서비스 코드는 그대로

```
1 UploadFile uploadFile = fileStore.store(image);  
2 user.changeProfileImage(uploadFile);  
3 // 1:다 도서 이미지도 각 BookImage.storedName에 키가 들어감
```


객체 삭제 요청

```
1 s3Client.deleteObject(builder → builder  
2     .bucket(properties.bucket())  
3     .key(storedName));
```

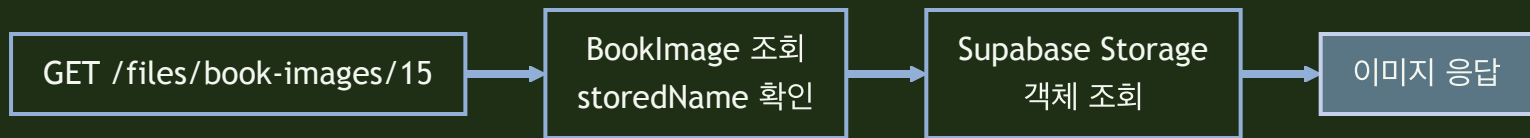
삭제할 때 주의할 점

- 자동 삭제 아님: DB 행을 지워도 스토리지 객체는 자동으로 지워지지 않음
- 복구 불가: Supabase Storage는 S3 버저닝을 지원하지 않아 삭제한 객체는 복구 불가
- 파일 교체 순서: 새 객체 저장과 DB 변경이 성공한 뒤 이전 객체를 삭제

자체 컨트롤러로 이미지를 제공하는 이유

- 접근 불가: 비공개 버킷의 객체는 공개 URL로 열 수 없음
- 키 은닉: URL에는 객체 키 대신 DB의 이미지 ID만 노출
- 판단 주체: 접근 권한을 애플리케이션이 판단할 수 있음

이미지 요청이 서버를 거치는 흐름



파일 조회 컨트롤러의 기본 뼈대

```
1  @Controller
2  @RequestMapping("/files")
3  public class FileViewController {
4
5      private final BookImageRepository bookImageRepository;
6      private final S3Client s3Client;
7      private final StorageProperties properties;
8
9      // 생성자 주입 생략
10 }
```

이미지 ID로 객체를 찾아 스토리지에서 읽기

```
1  @GetMapping("/book-images/{id}")
2  public ResponseEntity<Resource> getBookImage(@PathVariable Long id) {
3      BookImage image = bookImageRepository.findById(id)
4          .orElseThrow(() -> new ResponseStatusException(NOT_FOUND));
5      ResponseInputStream<GetObjectResponse> input = s3Client.getObject(b -> b
6          .bucket(properties.bucket()).key(image.getStoredName()));
7      // 다음 슬라이드에서 응답 생성
8  }
```

읽은 객체를 응답으로 보내내기

```
1 String raw = input.response().contentType();
2 MediaType type = raw == null
3     ? MediaType.APPLICATION_OCTET_STREAM
4     : MediaType.parseMediaType(raw);
5
6 return ResponseEntity.ok()
7     .contentType(type)
8     .contentLength(input.response().contentLength())
9     .body(new InputStreamResource(input));
```

스트리밍으로 응답하는 이유

- 메모리 절약: `InputStreamResource` 를 쓰면 파일 전체를 메모리에 올리지 않고 전달
- 진행률 제공: `contentLength`를 함께 지정해 브라우저가 진행률을 알 수 있음
- 동일 패턴 재사용: 프로필 이미지도 `/files/users/{id}/profile-image` 처럼 같은 방식으로 구현

비공개 자료라면 권한 확인이 먼저

- 소유자 검증: 객체를 읽기 전에 로그인 사용자와 파일 소유자가 같은지 확인
- 추측 접근 차단: 확인 없이 ID만으로 응답하면 URL 추측으로 열람 가능
- 404로 응답: 권한이 없으면 존재 여부도 알리지 않도록 동일한 오류로 처리

타임리프에서 자체 컨트롤러 주소 사용

```
1 
```

자체 컨트롤러 방식의 대가

- 은닉된 내부 호출: 개발자 도구에는 애플리케이션 요청만 보이지만 서버 내부에서는 Supabase를 호출
- 중계 부담: 모든 이미지 바이트가 애플리케이션 서버를 거쳐 네트워크 부담이 커짐
- 단계적 접근: 이번 차시는 흐름 이해를 위해 이 방식을 우선

공개 범위와 파일 제공 방식

구분	적합한 파일	제공 방법
공개 버킷	누구나 볼 수 있는 이미지	스토리지 공개 URL
비공개 버킷	회원 파일(이번 실습)	자체 컨트롤러 또는 서명 URL

- **중계 생략:** 서버가 파일을 중계하는 대신 권한 확인 후 짧은 만료 시간의 서명 URL을 발급
- **직접 다운로드:** 브라우저가 스토리지에서 직접 내려받아 서버 부담이 줄어들
- **적용 범위:** 이번 차시에서는 개념만 확인하고 구현은 자체 컨트롤러로 진행

프리티어에서 아껴 쓰기

- 고아 객체 정리: 1GB는 원본 이미지가 쌓이면 빠르게 소진되므로 DB가 참조하지 않는 객체를 정리
- 전송량 관리: 같은 이미지를 반복 조회해도 전송량이 소모되므로 목록 화면의 이미지 크기·개수를 제한
- 이중 제한: 애플리케이션 5MB 제한과 버킷 제한을 함께 걸어 두 단계로 차단

업로드 실패와 트랜잭션 경계

- 실패 가능성: 업로드는 네트워크 요청이라 실패할 수 있고 실패하면 DB에 키를 저장하지 않고 재시도를 안내
- 분리된 트랜잭션: 스토리지 업로드와 DB 트랜잭션은 하나로 묶이지 않음
- 주기적 정리: 업로드 후 DB 저장이 실패하면 참조되지 않는 객체가 남으므로 주기적으로 정리

학습 요약

- **저장 구조:** 실제 파일은 객체 스토리지에, 객체 키는 DB에 저장
- **연결 선택:** AWS SDK 직접 설정과 Spring Cloud AWS 스타터 중 하나를 선택
- **보안 원칙:** 서버 전용 S3 키를 환경 변수로 관리하고 접근 권한을 확인
- **파일 응답:** 자체 컨트롤러로 중계하고 트래픽 증가 시 서명 URL을 고려